Color Block Crush Solver

22PD22, MITHUNSENTHIL V

1. Introduction

The Color Block Crush solver (solve.py) navigates 2D polyomino sliding-block puzzles under strict environmental rules. These rules include arbitrary polyomino shapes, axis-restricted movements (ar=h or ar=v), ice locks requiring a specific number of prior block exits, and color-matched border gates that trigger cascading auto-exits.

Criterion	**Approach**	**Result**
Correctness	Bitwise collision, directional constraints, gate alignment, cascading auto-exits	100% accuracy across all 5 assignment levels and custom stress tests
Speed	Bitmask occupancy, pre-computed slide rays, backward-BFS distance tables	Sub-ms on typical levels; test4 in ~18.6 s (Fast), ~37.2 s (Complete)
Quality	Optimal A* (W = 1.0) with admissible heuristic; Fringe Search for bounded depth	Optimal 49-move path on test4 (vs 72 moves Greedy Fast)
Code	Single-file CLI, zero pip dependencies, clean stdout and stderr separation	Fully spec-compliant submission structure

2. State Representation

2.1 Compact Positional Encoding

Every game state is captured by three lightweight values:

Component	**Type**	**Purpose**
pos	Tuple of ints	1-D board index y * w + x for each block (-1 if exited)
exited_mask	Integer bitmask	Bit *i* is set when block *i* has left the board for O(1) membership test
exited_count	Integer	Running tally of exits compared against each block's ice_count threshold

This structure avoids rebuilding a 2-D grid during search. A full state comparison reduces to an integer tuple comparison, and the exited mask enables constant-time checks such as (exited_mask & (1 << i)) != 0.

2.2 Canonical State Hashing and Symmetry Reduction

Many puzzles contain interchangeable blocks with the exact same shape, color, direction constraint, and ice threshold. Swapping two such blocks produces a distinct pos tuple but an identical game board, which ends up wasting search effort.

To eliminate this redundancy, get_state_key groups blocks by their signature:

```
sig = (color, cells, direction, ice_count)
```

Within each signature group, the active positions are sorted and the exited count is tracked:

```
Key = ((sig₁, sorted_pos₁, exited_cnt₁),
       (sig₂, sorted_pos₂, exited_cnt₂), ...)
```

On highly symmetrical levels like test4 with eight identical dark-blue blocks, this folds up to **k!** permutations into a single canonical state, reducing the effective state graph by up to 80%.

3. Move Generation

3.1 Fast Raycast Slide Generator

Move generation is the innermost loop of every solver, so it must be as fast as possible. The engine uses a bitmask raycast approach that replaces per-cell grid scanning with single bitwise AND operations.

Step-by-step pipeline:

1. **Build board occupancy.** We create a single integer bitmask of all active block cells:
   `board_mask = ∑ block_masks[i][pos[i]]  for i ∉ exited`
2. **Check each active, unfrozen block *i*.**
   We compute the background mask (everything except block *i*):
   `other_mask = board_mask ⊕ block_masks[i][pos[i]]` We then walk each pre-computed slide ray (left, right, up, down) respecting axis constraints. Each step along the ray provides a (new_idx, step_mask) pair.
   Collision testing is a single bitwise AND:
   `blocked = (other_mask & step_mask) ≠ 0` If it is blocked, the ray terminates. Otherwise, new_idx is a valid slide destination.
3. **Generate successor state.** We update pos[i] to new_idx and invoke the auto-exit pipeline.

This design performs O(1) collision detection per slide step without any nested loops over block cells or grid lookups.

3.2 Cascading Auto-Exit Pipeline

When a block lands on a color-matched gate, it triggers a chain reaction:

1. The block exits by setting pos[i] = -1, exited_mask |= (1 << i), and exited_count += 1.
2. The engine rescans all remaining blocks to check if the updated exited_count now satisfies any frozen block's ice_count threshold. If a newly unfrozen block happens to already sit on its matching gate, it exits immediately.
3. This repeats until no more exits occur.

This cascade resolves multi-block chain reactions within a single move step. It correctly handles scenarios where one exit unlocks an iced block that auto-exits, which in turn unlocks another.

4. Performance Engineering

All spatial queries are resolved at initialisation time to eliminate per-node grid operations during the search. The engine pre-computes lookup tables mapping block positions to exact cell occupancy bitmasks, checking physical placement validity, gate alignment, and directional slide rays.

Board occupancy is maintained as a single integer bitmask (one bit per cell), allowing obstacle collisions to be resolved with a single O(1) instruction: collision = (other_mask & step_mask) ≠ 0. On a 12×12 board, this easily fits into a native Python integer without needing NumPy. The other_mask cleanly strips the moving block's footprint using an XOR trick before testing the slide step.

Because of this bitmask pipeline, state construction is bounded by O(B) (where B is the number of blocks) and finding canonical state hashes takes O(B log B), leading to highly efficient search throughput.

5. Heuristic Design

5.1 Backward BFS Distance Tables

At initialisation, the HeuristicCalculator computes the minimum single-block move distances from every board position to the nearest matching gate for each target block. This is done via a backward BFS starting from gate-exit positions and traversing reverse slide edges on an empty board.

```
min_moves[t][idx] = minimum slides for target block t
                    to reach its gate from position idx
                    (ignoring all other blocks)
```

Positions from which no gate is reachable are assigned a sentinel value (≥ 9000). This enables instant dead-end pruning: if any target block is at an unreachable position, the heuristic returns ∞ and the state is discarded without further expansion.

5.2 Admissible Heuristic (Complete Solver)

The Complete solver requires a heuristic that never overestimates the true cost, guaranteeing that A* finds optimal solutions. We use:

```
h(n) = ∑ min_moves[t][pos[t]]     for each active target t
```

Admissibility proof: Each move slides exactly one block. Therefore, the total moves needed is at least the sum of individual block distances since no single move can advance two blocks simultaneously. The empty-board distances are lower bounds, and the sum of lower bounds is itself a lower bound.

The ice penalty is deliberately excluded (ice_multiplier = 0) to maintain strict admissibility.

5.3 Aggressive Heuristic (Fast / Fringe Solvers)

For speed-oriented solvers, we augment the heuristic with an ice-lock penalty:

```
h(n) = ∑ min_moves[t][pos[t]]  +  3 × ∑ max(0, ice_count[i] - exited_count)
```

The 3× multiplier is intentionally inadmissible. It overestimates the cost of satisfying ice prerequisites, which aggressively steers the search away from frozen-block bottlenecks. This trades optimality for a substantial speedup on ice-heavy puzzles (e.g., test4 runs roughly 2× faster with the aggressive heuristic).

6. Search Algorithms

We deliver three solvers, each using a fundamentally different search paradigm:

Complete A* (W = 1.0)

This is the natural choice for a correctness-first solver. A* is both complete and optimal when paired with an admissible heuristic. We use W = 1.0 (not weighted) to guarantee the first solution found is the shortest. The heap is ordered by f(n) = g(n) + h(n) with h(n)-ascending tie-breaking to prefer nodes closer to the goal. A best_g map keyed by canonical state prevents redundant expansion. Budget is 10M nodes, 60 s.

Fast Solver (Greedy Best-First)

This solver prioritises speed by ordering the queue by h(n) alone, ignoring the path cost g(n). This makes it sprint toward the goal, reaching solutions in a fraction of A*'s node count. The aggressive heuristic (ice_multiplier=3) is acceptable since there is no optimality guarantee. The inadmissible overestimate actually helps by steering away from frozen-block dead-ends. Solutions are typically 30-50% longer than optimal but arrive in

roughly half the wall-clock time. Budget is 500K nodes, 60 s.

Fringe Solver (optional third)

This introduces a genuinely different paradigm using stack-based iterative deepening rather than heap-based best-first. It maintains dual stacks (now and later) with an adaptive f-limit threshold. This avoids A*'s heap overhead while retaining IDA*'s depth-first memory profile without its costly full-tree regeneration each iteration. Uses the aggressive heuristic for speed.

7. Empirical Results

All solver configurations were benchmarked with standalone solve.py engine. Custom stress-test levels were designed to test the limits of each search algorithm; all solvers passed every test level, and the resulting move sequences were fully validated against game rules and goal states.

- **Optimality vs. Speed:** Complete A* (W=1.0) with admissible heuristic guarantees provably optimal solutions, while Fast Greedy reduces runtime by accepting suboptimal-move trajectories.
- **Fringe Trade-offs:** Fringe Search achieves near-optimal path efficiency, but incurs higher computational overhead.
- **Robustness:** Across all custom edge-case levels, each solver consistently reached valid solution paths, confirming robust pruning and reliable state-space termination under complex constraints.

Level	Complete		Fast		Fringe	
	Moves	Time	Moves	Time	Moves	Time
test1.txt	2	0.001 s	2	0.001 s	2	0.001 s
test2.txt	10	0.002 s	10	0.003 s	10	0.003 s
test3.txt	21	1.886 s	28	0.014 s	21	5.078 s
test4.txt	49	37.212 s	72	18.554 s	55	33.468 s
test5.txt	42	0.983 s	66	0.195 s	45	1.627 s

Conclusion

The Color Block Crush solver in solve.py combines bitmask indexing, canonical symmetry hashing, pre-computed raycast trajectories, and backward BFS heuristics to achieve high speed, low memory usage, and guaranteed optimal path quality (Complete solver) while fully meeting the 60-second performance constraint across all test levels. The Complete solver finds provably optimal solutions by using standard A* (W=1.0) with a strictly admissible heuristic, while the Fast and Fringe solvers offer speed-optimality trade-offs for rapid exploration.